Excel with OfficeCLI

This reference describes the OfficeCLI route. For tool alternatives, see tool selection. Resolve a working absolute path with environment setup before any `officecli` example below; then use that printed path instead of a bare `officecli`.

⚠️ Help-First Rule

This skill teaches what good xlsx looks like, not every command flag. When a property name, enum value, or alias is uncertain, consult help BEFORE guessing.

Bash

```
officecli help xlsx                          # List all xlsx elements
officecli help xlsx <element>                # Full element schema (e.g. pivotta
officecli help xlsx <verb> <element>         # Verb-scoped (e.g. add chart, set
officecli help xlsx <element> --json         # Machine-readable schema
```

Help reflects the installed CLI version. When this skill and help disagree, **help is authoritative**.

Shell & Execution Discipline

Shell quoting (zsh / bash). Excel paths contain `[]`, and number formats contain `$`. Both are shell metacharacters. Rules:

- ALWAYS quote element paths: `"/Sheet1/row[1]"`, not `/Sheet1/row[1]`.
- Use **single quotes** for any prop value containing `$`: `numFmt='$#,##0'`.
- For formulas with cross-sheet `!` references, use `batch` with a `<<'EOF'` heredoc (see Known Issues).
- `\n` and `\t` in a prop value ARE interpreted by the CLI — `\n` is a real in-cell line break (pair with `--prop wrapText=true`), `\t` a tab — consistent across xlsx / docx / pptx. Double them (`\\n`) for a literal backslash-n (rarely wanted). (`$` is the shell layer above — single-quote it.)

Incremental execution. Run commands one at a time and read each exit code. `officecli` mutates the file on every call; a 50-command script that fails at command 3 will cascade silently. One command → check output → continue.

Requirements for Outputs

Before reaching for a command, know what a good xlsx looks like. These are the deliverable standards every workbook MUST meet.

All Excel files

Zero formula errors. Every delivered workbook MUST have ZERO `#REF!`, `#DIV/0!`, `#VALUE!`, `#NAME?`, `#N/A`. No exceptions — guard denominators with `IFERROR` or `IF(x=0,...)`.

Formulas, not hardcoded values. If a number can be computed from other cells, it is a formula. Hardcoding `5000` where `=SUM(B2:B9)` belongs breaks the contract that the workbook stays live when inputs change. This is the single most important rule in this skill.

Professional font. Use one consistent, professional font across the workbook (Arial / Calibri / Times New Roman). Don't mix four fonts because one sheet came from CSV.

Explicit widths. There is no auto-fit. Any column the user will read MUST have `width` set — default 8.43 chars clips everything. Sensible starts: labels 20-25, numbers 12-15, dates 12, short codes 8-10.

Preserve existing templates. When editing a file that already has a look, match it. Existing conventions override these guidelines.

Visual delivery floor (applies to EVERY workbook)

Before you declare done, run `officecli view "$FILE" html -o "${FILE}.html"` and inspect the generated HTML file to confirm all of these:

- **No `###` in any cell.** `###` means a column is too narrow for its widest value. Every column the user reads needs an explicit `width`. `###` in a delivered file is unfinished work, never "a small visual nit".
- **No truncated titles.** Sheet titles, section headers, long labels must fit. Widen the column or apply `wrapText=true` on the cell.
- **No placeholder tokens rendered as data.** `$fy$24`, `{var}`, `<TODO>`, `xxxx` must never appear in a cell, chart title, series name, or legend. These are build-time tokens that escaped replacement.
- **Pie / doughnut slices have distinct fill colors.** If the slices render same-colored, switch to `bar` / `column` or set `colors=...` explicitly.
- **No empty trailing pages / empty chart anchors.** `anchor=D2:J18` over empty source cells looks like a broken chart.

If any of the above fails, STOP and fix before declaring done.

Print layout. Any sheet the user may print or send as a board pack needs page setup. Default portrait + no fit-to-page splits wide tables and charts mid-way. Pick the fit mode by

sheet shape:

```bash
# Summary / chart / dashboard sheet (small, ≤ ~40 rows): fit to a single page.
officecli set "$FILE" "/Summary" --prop orientation=landscape --prop fitToPage=
# Tall data table (dozens+ rows): fit WIDTH only, let height paginate naturally
# fitToPage=true here crushes every row onto one page → unreadable (### dates,
officecli set "$FILE" "/Data" --prop orientation=landscape --prop fitToPage=1x0
```

`fitToPage=true` == `1x1` == fit both axes to one page — correct only when the sheet is already short. `1x0` = fit 1 page wide, unlimited pages tall. Trigger: sheet holds a chart, or > 8 columns, or the user's ask mentions print / board / investor.

Financial models only — skip this section if you are building a template, tracker, CSV import, or operational sheet

Scope: budgets, forecasts, 3-statement models, valuation, any `$`-heavy analytical workbook. A customer-support tracker or onboarding template does not need this section.

Color coding — industry standard. Five core colors used as a language, not decoration. A reviewer should tell what a cell IS by color alone — before reading the formula.

Color	Role	Example
Blue text `0000FF`	Hardcoded inputs, scenario variables	`font.color=0000FF`
Black text `000000`	ALL formulas and calculations	default
Green text `008000`	Cross-sheet links inside this workbook	`font.color=008000`
Red text `FF0000`	Links to external files / workbooks	`font.color=FF0000`
Yellow fill `FFFF00`	Key assumptions needing review	`fill=FFFF00`

A reviewer should tell what a cell IS just by its color — before reading the formula. This is a communication contract, not a cosmetic preference.

Number formatting — standards, not preferences.

- **Years** are text, not numbers. Format `2026` not `2,026` — use `numFmt="@"` or set `type=string`.

- **Currency** carries its unit in the header (`Revenue ($mm)`), not in every cell.
- **Zeros display as** `-` , not `0` . Use `$#,##0;($#,##0);"-"` .
- **Percentages** default to one decimal: `0.0%` .
- **Negatives use parentheses**: `(1,234)` not `-1,234` .
- **Valuation multiples** use `0.0x` format (EV/EBITDA, P/E, etc.).

Assumptions live in cells, not inside formulas. `=B5*(1+$B$6)` is correct; `=B5*1.05` is a bug. Document each blue hardcoded input with an adjacent source note in the next cell or a cell comment:

```
Plain Text

Source: Company 10-K, FY2024, Page 45, Revenue Note
Source: Bloomberg, 2026-05-02, AAPL US Equity
Source: Management guidance, Q2 2026 earnings call
```

Any hardcoded number without a source is an undocumented assumption — a reviewer cannot audit it.

Common Workflow

Six steps. Every non-trivial build follows this shape.

1. **Open/save lifecycle.** Use `officecli open <file>` at the start and `officecli save <file>` at the end to flush to disk — `save` only writes and leaves the resident warm for follow-up edits; reach for `officecli close <file>` only to release the resident on a one-shot handoff. Both are always safe (never error or lose work). For many cells, use `batch` : **⩽ 50 ops/block recommended; tested up to 80+ ops per block on pure value-set payloads with zero failures.** Cross-sheet formulas still need a `<<'EOF'` heredoc so `!` is not shell-escaped (see Known Issues); they can run through a resident. **Flush only at the non-officecli boundary:** officecli's own reads always see your edits; run `save` / `close` only before a non-officecli program reads the file (openpyxl/pandas, Excel, a renderer, delivery).
2. **Create or load.** `officecli create "$FILE"` (new) or `officecli view "$FILE" outline` (existing — get the lay of the land first).
3. **Build incrementally.** One command, read the output, continue. After any structural op (new sheet, chart, named range, pivot), run `get` on it to confirm shape before stacking more on top.
4. **Format.** Column widths, number formats, freeze panes, tab colors, header fills. Formatting is not optional polish — per "Requirements for Outputs" it is part of the

deliverable.

5. **Save, then reckon with the cache.** `officecli save <file>` writes to disk. Newly-added formulas ship without cached values; when a human opens the file in a spreadsheet app, the app recalculates and populates them. **But your downstream `INDEX/MATCH`, `SUMPRODUCT`, or any formula that references an upstream formula will cache whatever the upstream cached at write-time — often `0` or a stale value — and that cached lie survives into non-recalculating readers.** After any multi-formula build involving array formulas (`SUMPRODUCT`, `SUMIFS` with dynamic criteria) or cross-sheet chains, **re-touch every downstream cell** (run `set` again with the same formula) so the engine recomputes its cache from the freshly-cached upstream. ⚠️ Re-touch on cross-sheet chains via resident is unreliable (see Batch / resident caveats) — prefer non-resident `set` for the re-touch pass. Then `officecli get` a few downstream cells and eyeball that their `cachedValue=` is plausible. `validate` is safe with a resident open and itself flushes pending edits to disk (same as docx / pptx).
6. **QA — assume there are problems.** See the QA section. You are not done when your last command exited 0; you are done after one fix-and-verify cycle finds zero new issues.

Quick Start

Minimal viable xlsx: 3 months of revenue + a total formula + column widths + a currency format. Adapt, don't copy-paste — your file, your data.

Bash

```
FILE="data.xlsx"
officecli create "$FILE"
officecli open "$FILE"
officecli set "$FILE" /Sheet1/A1 --prop value=Month --prop bold=true
officecli set "$FILE" /Sheet1/B1 --prop value=Revenue --prop bold=true
officecli set "$FILE" /Sheet1/A2 --prop value=Jan
officecli set "$FILE" /Sheet1/A3 --prop value=Feb
officecli set "$FILE" /Sheet1/A4 --prop value=Mar
officecli set "$FILE" /Sheet1/B2 --prop value=42000 --prop numFmt='$#,##0'
officecli set "$FILE" /Sheet1/B3 --prop value=45000 --prop numFmt='$#,##0'
officecli set "$FILE" /Sheet1/B4 --prop value=48000 --prop numFmt='$#,##0'
officecli set "$FILE" /Sheet1/A5 --prop value=Total --prop bold=true
officecli set "$FILE" /Sheet1/B5 --prop formula="SUM(B2:B4)" --prop bold=true -
officecli set "$FILE" "/Sheet1/col[A]" --prop width=12
officecli set "$FILE" "/Sheet1/col[B]" --prop width=15
officecli close "$FILE"
officecli validate "$FILE"
```

Verified: `validate` returns `no errors found`, `B5` resolves to `135000`. This is the shape of every build: open → set cells/formulas → format → close → validate.

CSV / bulk import

CSV/TSV are import inputs, not native documents that `officecli view data.csv text` can open. Import them into an XLSX workbook before using OfficeCLI's document operations.

For PDF delivery, set the workbook's column widths and print layout, then see PDF conversion for converter options and verification.

Native `import` command (preferred for CSV/TSV). Fastest path; loads a CSV into a sheet in one call. `--header` sets AutoFilter + freeze pane on row 1. Widths and `numFmt` still need a follow-up pass.

Bash

```
officecli import "$FILE" /Sheet1 --file data.csv --header
officecli import "$FILE" /Sheet1 --file data.tsv --format tsv --header
officecli import "$FILE" /Sheet1 --stdin --start-cell B2 < data.csv
```

Python + batch fallback — use when you need custom type coercion, formula injection, or the CSV lives inside another data pipeline. Recipe for 600-6000+ cells:

Python

```
# gen_batch.py — produces batch chunks of 80 value-set ops each
import csv, json
ops = []
with open("data.csv") as f:
    reader = csv.reader(f)
    for r, row in enumerate(reader, start=1):
        for c, val in enumerate(row):
            col = chr(ord('A') + c)
            ops.append({"command":"set","path":f"/Data/{col}{r}",
                        "props":{"value": val}})
for i in range(0, len(ops), 80):
    print(json.dumps(ops[i:i+80]))
```

Bash

```
python gen_batch.py | while IFS= read -r chunk; do
  printf '%s\n' "$chunk" | officecli batch "$FILE"
done
```

Outcome: 648-row retail CSV (6490 cells) loads in ~30s, zero failures. Tune: start at 80 ops/chunk, drop to 40 if any chunk fails. Numeric type inference and formulas come later via targeted `set` — batch in this recipe is pure value injection.

Reading & Analysis

Start wide, then narrow. `outline` first tells you what sheets exist and where the data is; jump into `view` / `get` / `query` only once you know where to look.

Open the rendered workbook to eyeball your own work.

- `officecli view "$FILE" html` — Read the returned HTML to audit the rendered output. Each sheet is addressable, charts render inline. Catches `###`, placeholder leakage, pivot layout, row-height clipping.
- `officecli watch "$FILE"` keeps a live preview running for the human user — they open it at their own discretion. Use when the user wants to watch along; agent self-check uses `view html` above.
  Use `view html` as your **first visual check after a batch of edits** — fix at source. For final visual verification, the user opens the `.xlsx` in their Excel / WPS / Numbers viewer.

Orient. Sheets, dimensions, formula counts.

Bash

```
officecli view "$FILE" outline
```

Extract. Plain text dump for content QA or reading context; scope with `--start` / `--end` / `--cols` for big files.

Bash

```
officecli view "$FILE" text --start 1 --end 50 --cols A,B,C
```

Other `view` modes worth knowing: `annotated` (cell values + types/formulas + warnings), `stats` (numeric summaries), `issues` (broken formulas, empty sheets, missing refs).

Round-trip dump. `officecli dump "$FILE" [path]` serializes the workbook — or one worksheet (`/Sheet1`, `/sheet[N]`) — into a replayable batch JSON; `officecli batch new.xlsx --input dump.json` replays it. Use it to learn from an existing workbook's structure or clone/adapt a template instead of reading raw OOXML. Coverage per `dump --help`; subtree dumps don't carry workbook-level resources (settings, named ranges) — the replay target must already define them.

Bash

```
officecli dump "$FILE" -o blueprint.json            # whole workbook
officecli dump "$FILE" /Sheet1 -o sheet.json        # one worksheet
officecli batch new.xlsx --input blueprint.json
```

Inspect one element. Use XPath-style paths. Always quote — shells glob `[N]`.

Bash

```
officecli get "$FILE" "/Sheet1/A1"            # one cell
officecli get "$FILE" "/Sheet1/A1:D10"        # range
officecli get "$FILE" "/Sheet1/chart[1]"      # chart
officecli get "$FILE" "/Sheet1/table[1]"      # ListObject
officecli get "$FILE" "/namedrange[1]"        # workbook-level named range
```

Add `--depth N` to expand children; add `--json` for machine output. Full element list: `officecli help xlsx`.

Query across the workbook. CSS-like selectors. Use for systematic checks (formula coverage, error cells, empty headers) rather than hand-walking.

Bash

```
officecli query "$FILE" 'cell:has(formula)'       # every formula cell
officecli query "$FILE" 'cell:contains("#REF!")'  # broken references
officecli query "$FILE" 'cell[type=Number]'       # typed filter
officecli query "$FILE" 'Sheet1!B[value!=0]'      # sheet-scoped
```

Operators: `=`, `!=`, `~=` (contains), `>=`, `<=`, `[attr]` (exists).

Merge cells shortcut. `officecli query "$FILE" merge` or `mergedrange` — both are aliases for `mergeCell`. Returns every merged range in the workbook without hand-walking `<mergeCell>` entries.

When the data is big enough that a row-walk is useless, reach for Excel's own analytical elements:

- Build a **pivot table** with `officecli add` (`--type pivottable`) to group/aggregate without writing 20 SUMIFs. Attach a **slicer** (`--type slicer`) to give the reader a filter UI.
- Drop a **sparkline** (`--type sparkline`) in a row to show per-row trends — cheaper than one line chart per row and they print inline. `type` is a strict enum: **`line` | `column` | `stacked`** (plus aliases `winloss` / `win-loss` → `stacked`). Invalid `type=` values hard-fail — no silent fallback to `line` anymore.
- Run `officecli help xlsx pivottable`, `officecli help xlsx slicer`, `officecli help xlsx sparkline` for the exact prop names.

Creating & Editing

Ninety percent of a build is cells, formulas, formatting, and one or two charts. The verbs: `add` (new element), `set` (change a prop), `remove`, `move`, `swap`, `batch`.

Cells and formulas

Set a value and its format in one call. Never write `=` at the start of a formula — the CLI strips it.

Bash

```
officecli set "$FILE" /Sheet1/B5 --prop formula="SUM(B2:B4)" --prop numFmt='$#,
officecli set "$FILE" /Sheet1/C5 --prop formula="B5/A5" --prop numFmt="0.0%"
```

Structural properties (width, height, freeze, tabColor) live on row / col / sheet nodes:

Bash

```
officecli set "$FILE" "/Sheet1/col[A]" --prop width=20
officecli set "$FILE" "/Sheet1/row[1]" --prop height=22
officecli set "$FILE" "/Sheet1" --prop freeze=A2 --prop tabColor=1F4E79
```

Named ranges

Prefer named ranges over `$B$6` in formulas. They self-document (`GrowthRate` beats `$B$6`) and they let you move the assumption cell without breaking formulas. Because `ref` values contain both `!` and `$`, add them through a batch heredoc:

Bash

```
cat <<'EOF' | officecli batch "$FILE"
[
  {"command":"add","parent":"/","type":"namedrange","props":{"name":"GrowthRate
]
EOF
```

See `officecli help xlsx namedrange` for the full schema.

Batch JSON does NOT accept shell aliases. Inside batch `props`, always use the full dotted name — `"font.color": "FF0000"`, `"font.size": 14`, never `"color": "FF0000"` (ambiguous: text vs fill). On a bare cell, even the shell form is rejected: `--prop color=1F4E79` errors with `ambiguous in cell context — use 'font.color' (text) or 'fill' (bg)`. Rule: in any batch JSON or cell prop, write

`font.color` / `fill` explicitly. `parent` should be `"/"` for workbook-level elements and `"/SheetName"` for sheet-scoped; empty string is not equivalent.

Charts

Chart types live under `officecli help xlsx chart` — the enum is long (20+). Pick the right one for the message: column for category comparison, line for time series, pie only when slices are self-evidently proportional, scatter for correlation. Avoid exotic types unless they answer a specific question.

Three ways to feed chart data. Pick one per chart — mixing them at add-time is a common trap.

Form	Shape	When to use
(a) inline `data`	`--prop data="Sales:100,200,300" --prop categories="Jan,Feb,Mar"`	Tiny demo charts, numbers you will not edit. Source of truth lives in the chart XML, not a cell.
(b) 2D `dataRange`	`--prop dataRange="Sheet1!A1:B4"` (first col = categories, first row = header / series name)	Normal case. Must be **2-D** — single column fails with "Chart requires data".
(c) dotted per-series	`--prop series1.name=Sales --prop series1.values="Sheet1!B2:B4" --prop series1.categories="Sheet1!A2:A4"`	Multi-series charts where each series points at non-contiguous ranges, or you want explicit series naming. `series1.values` alone (no `categories`) emits a chart with `1,2,3` as the x-axis.

The single-column trap. `dataRange="Sheet1!B2:B13"` looks like "value column" but the engine rejects it with `Chart requires data`. Either widen the range to include the category column (`A2:B13`), or switch to form (c) with explicit `series1.categories`.

Move / resize a chart after create: `set chart[N] --prop anchor="F5:N25"` (also `--prop x= --prop y= --prop width= --prop height=`). **Series are still immutable** — to add/change a series, `officecli remove` the chart and `officecli add` with the full series list. Note `remove chart[1]` shifts `chart[2]` → `chart[1]` and re-add **appends at the end** — to preserve chart order, remove all and rebuild in order.

Anchor sizing. No auto-fit. A column chart with 5-6 categories + 2 series needs roughly `A5:L22` (12 cols × 18 rows) to show all labels uncut. Narrower and X-axis labels clip; wider and the chart can split across pages on print/export. If in doubt, start narrow, preview via

`view html` (inspect the HTML output), widen in increments. Page layout (below) is the other half of the fix.

Chart `dataRange` — always prefix with the sheet. Even when the chart lives on the same sheet, write `dataRange="Summary!A17:C22"`, not `A17:C22`. The sheet-less form works inconsistently; the prefixed form is 100% reliable.

officecli adds extended chart types the classic Excel object model lacks: `boxWhisker`, `waterfall`, `funnel`, `histogram`, `treemap`, `sunburst`, `pareto`. Use them when the data calls for them.

NEVER put unreplaced template tokens in chart title / series name / legend / axis title. `$fy$24`, `{var}`, `<TODO>`, `$VAR`, `{{placeholder}}` render **literally** in the legend — validate passes, but a CFO sees `$fy$24` where "FY2024" should be. Always bind to final text or a cell reference (`title="FY2024 Revenue"` or `series1.name="Sheet1!A1"`).

Conditional formatting

Three common flavors, each with its own prop shape (consult `officecli help xlsx cf`):

- **Color scales**: cells shaded on a gradient by value — `type=colorscale` with `minColor` / `midColor` / `maxColor`.
- **Data bars**: in-cell bars showing magnitude — `type=databar`. Set explicit `min` / `max` for consistent scaling across a column; defaults are valid if you omit them.
- **Formula rules** (the `formulacf` element): highlight row when a condition is true — `type=formula` with `formula="$C2>1000"` and a fill/font.

Rule: apply CF sparingly. A workbook where every cell is colored tells the reader nothing.

Data validation

Input cells in trackers and templates MUST carry data validation. It's cheap and it stops entire classes of downstream bugs. **Three list-source patterns** — pick based on where the allowed values live.

(a) Inline list — allowed values are short and fixed in the rule itself.

Bash

```
officecli add "$FILE" /Sheet1 --type validation \
  --prop sqref="C2:C100" --prop type=list \
  --prop formula1="Yes,No,Maybe" \
  --prop showError=true --prop errorTitle="Invalid" --prop error="Select from l
```

(b) Named range (preferred for cross-sheet lookups) — allowed values live in another sheet and may grow. Define the named range first, then reference it. Use a batch heredoc

because `ref` contains `!` and `$`:

```bash
cat <<'EOF' | officecli batch "$FILE"
[
  {"command":"add","parent":"/","type":"namedrange","props":{"name":"StatusList
  {"command":"add","parent":"/Sheet1","type":"validation","props":{"sqref":"B2:
]
EOF
```

(c) Direct cross-sheet range — no named range, raw `Lookups!$A$2:$A$4` inside `formula1`. Also needs a batch heredoc to keep `!` and `$` intact:

```bash
cat <<'EOF' | officecli batch "$FILE"
[
  {"command":"add","parent":"/Sheet1","type":"validation","props":{"sqref":"C2:
]
EOF
```

If you write the cross-sheet variant as `--prop formula1=...` on the shell, the `!` gets shell-mangled into `\!` and the dropdown will silently fall back to no list. Verify with `officecli get "$FILE" /Sheet1/validation[N]` — `formula1=` must show a plain `!`, no backslash.

Other common `type` values: `decimal`, `whole`, `date`, `textLength`, `custom`. See `officecli help xlsx validation` for operators and the full prop list.

Other elements (one-liners)

- **Tables** (ListObjects) — `add --type table` with a range; gives auto-filter + structured refs. `officecli help xlsx table`.
- **Comments** — `add --type comment`; use for documenting hardcoded assumptions. `officecli help xlsx comment`.
- **Sheet reordering** — `officecli move`, not `swap`. `swap` only works on row/cell paths.

Chart Axis-by-Role

Editing a chart axis in place is cheaper than rebuilding the chart. Address axes by **role** (`value` = Y, `category` = X), not by index — the XML order isn't stable.

```bash
```

```
officecli get "$FILE" "/Sheet1/chart[1]/axis[@role=value]"
officecli set "$FILE" "/Sheet1/chart[1]/axis[@role=value]" --prop min=0 --prop
officecli set "$FILE" "/Sheet1/chart[1]/axis[@role=category]" --prop title="Mon
```

Safe props: `title`, `min`, `max`, `majorGridlines`, `visible`, `labelRotation`.

QA (Required)

Assume there are problems. Your job is to find them.

Your first workbook is almost never correct. Treat QA as a bug hunt, not a confirmation step. If you found zero issues on first inspection, you were not looking hard enough. The formulas look fine **until** you check two of them against source cells.

Minimum cycle before "done"

1. `officecli view "$FILE" issues` — empty sheets, broken formulas, missing refs.
2. `officecli view "$FILE" annotated` (sample ranges) — values + types + warnings.
3. For every Excel error type, query it:

   Bash

   ```
   officecli query "$FILE" 'cell:contains("#REF!")'
   officecli query "$FILE" 'cell:contains("#DIV/0!")'
   officecli query "$FILE" 'cell:contains("#VALUE!")'
   officecli query "$FILE" 'cell:contains("#NAME?")'
   officecli query "$FILE" 'cell:contains("#N/A")'
   ```

4. `officecli validate "$FILE"` — safe with a resident open; `validate` flushes pending edits to disk itself.
5. **Visual pass — walk every sheet via the HTML preview.** Run `officecli view "$FILE" html -o "${FILE}.html"` and inspect the generated HTML file. Each sheet renders with charts inline. Scan for `###`, truncated titles, placeholder tokens (`$fy$24`, `{var}`, `<TODO>`), sliced charts, white-slice pie charts, empty chart anchors — **STOP and fix before declaring done**. "validate pass" is not delivery; "the preview looks like a real workbook" is delivery. For human preview, run `officecli watch "$FILE"` (user opens the live preview at their own discretion) or have them open the `.xlsx` directly in Excel / WPS / Numbers.
6. **Print layout fix (wide tables / multi-chart sheets).** When a sheet holds a chart or a wide table and the user will print it, set per-sheet page layout — but match the fit mode to the sheet's height:

Bash

```
# Short summary / chart sheet → fit to one page.
officecli set "$FILE" "/Summary" --prop orientation=landscape --prop fitToPa
# Tall data table → fit width only (fitToPage=true would crush all rows onto
officecli set "$FILE" "/Data" --prop orientation=landscape --prop fitToPage=
```

Outcome: charts/wide tables print without mid-chart splits; tall tables stay readable across natural page breaks. Apply to every sheet that holds a chart or a > 8-column table.

7. If anything failed, fix, then **rerun the full cycle**. One fix commonly creates another problem.

`officecli view issues` + `view html` are the structural QA pair: `issues` catches broken formulas and empty sheets; `view html` (inspect the HTML output) catches `###`, truncation, and token leakage. Chart fill colors / theme tints can vary across viewers — spot-check in the user's target viewer when color fidelity matters.

Formula verification checklist

- [ ] Pick 2-3 formulas at random. Run `officecli get` on each. Confirm the formula string is what you intended **and** `cachedValue=` is what you expect — arithmetic in your head.
- [ ] **Cached value sanity on every summary cell.** Any cell that aggregates (COUNTA / COUNTIF / SUMPRODUCT / INDEX&MATCH) must have a plausible `cachedValue`. If a progress tracker shows `199 / 199 / 100%` on a blank template, the cache is lying — re-touch the formula via `set` (forces recompute) or manually set a correct cached value. Do NOT ship "validate passes but the numbers are fiction".
- [ ] **Spot-check one cell per numeric column.** `%` columns showing integer `0.0%` throughout means the denominator is wrong or the numerator is cached stale — investigate one cell, fix the pattern.
- [ ] Ranges include every row: off-by-one on `SUM(B2:B12)` when data goes to `B13` is the most common bug.
- [ ] Cross-sheet formulas (`Sheet1!A1`) contain no `\!` . If `officecli get` shows `Sheet1\!A1` , the `!` was shell-corrupted — delete and re-enter via batch/heredoc.
- [ ] Named ranges (`officecli get "$FILE" "/namedrange[1]"`) point at what their names claim.
- [ ] Every `/` denominator is guarded — `IFERROR(x/y, 0)` or `IF(y=0, 0, x/y)` .

- [ ] Chart data vs source cells: for every chart with inline data, spot-check data points against `officecli get` of the source cells.
- [ ] Chart title / series name / legend contain **no** unreplaced tokens (`$...$` , `{var}` , `<TODO>`). Grep the chart via `officecli get /Sheet1/chart[N]` .

Template QA

When editing a template, check for leftover placeholders — they look like content and slip past `validate` :

Bash

```
officecli query "$FILE" 'cell:contains("{{")'
officecli query "$FILE" 'cell:contains("xxxx")'
officecli query "$FILE" 'cell:contains("TBD")'
```

Fresh eyes

When you finish a workbook, open it fresh. Read `view text` / HTML preview top-to-bottom as if you are a new reviewer — look for formulas, numbers that look off, formatting inconsistency, missing data.

Honest limit

`validate` catches schema errors, not design errors. A workbook can pass `validate` with every number wrong. The checklist above — especially spot-checking formulas against source cells — is how you catch what validation can't.

Known Issues & Pitfalls

The cross-sheet `!` trap (short)

Shells (bash history expansion, zsh splitting) and CLI arg parsing mangle `!` in `Sheet1!A1` into `\!` . A formula containing `\!` is silently broken — it renders as literal text and references nothing.

Fix. Use a batch heredoc with single-quoted delimiter (`<<'EOF'`), which disables all shell expansion:

Bash

```
cat <<'EOF' | officecli batch "$FILE"
[{"command":"set","path":"/Summary/B2","props":{"formula":"Revenue!B13"}}]
```

```
EOF
```

Verify. After writing, `officecli get` the cell; `formula=` must show a plain `!` with no backslash.

CLI bug backlog (short)

CLI constraints and gaps to work around — not defects in the output file.

- **Chart series are immutable after create** — to add/change a series: `remove` + `add` with the full series list. (Position is mutable: `set chart[N] --prop anchor=` / `x/y/width/height`.) `remove chart[N]` shifts subsequent indices down; re-add appends at end.
- **Cross-sheet formula batches run fine through a resident** — a prior "deadlocks even at 3-5 ops" caution no longer reproduces. Pure value-set batches stay reliable at 50-80+ ops too. If you ever hit a hang, fall back to a non-resident one-big-batch or individual `set`. **Multiple resident processes on the same file/machine can still contend** — expect non-deterministic hangs if another agent/session holds a resident on the same file.
- **Conditional formatting naming asymmetry** — the element name for `--type` is `conditionalformatting`; the path suffix is `/cf[N]`. Use `officecli help xlsx conditionalformatting` for schema, `/cf[N]` for paths.
- **Sheet `position` prop on add** — help says Add processes `position`, but the prop is often ignored. Reorder with `officecli move --index` / `--after` / `--before` after creating the sheet.
- **`remove /sheet[N]` cascade guard** — rejects sheet remove/rename when the sheet is referenced by validation / conditional format / sparkline / hyperlink / named range on another sheet. Remove those dependent elements first, then remove the sheet.
- **Batch JSON rejects cell `color` alias** — inside batch `props`, `"color": "FF0000"` errors `ambiguous in cell context — use 'font.color' (text) or 'fill' (bg)`. The CLI at shell level accepts `--prop color=...` / `--prop size=14` as aliases on non-cell elements, but inside batch JSON on a cell always write the full dotted name: `"font.color"`, `"font.size"`, `"font.name"`.

Renderer caveats (cross-viewer color fidelity)

`officecli view html` is the right tool for structural QA (overflow, truncation, placeholder leakage, layout) — inspect the HTML output. Some chart rendering details vary across the viewer the end user opens the file in. Observed divergences:

- **Pie / doughnut fill colors may collapse to a single theme tint** in some viewers (slices look "all white" or "all one color"). The file may be fine in the user's target viewer.
- **Line chart / column chart series colors may drift** from the workbook theme in some viewers.

- **Form-control checkboxes may render as double-boxed** in some viewers.

Before calling a color or chart "broken", open the file in the user's actual target viewer. If it looks correct there, the problem is viewer rendering, not data — do not chase it. The CLI's structural checks (`###` , truncation, placeholder text, layout) remain authoritative.

Escape layers (shell quoting is above; these are the extras)

`$` is the shell layer (single-quote it, above). `\n` / `\t` in a prop value ARE interpreted by the CLI into a real newline / tab. Two more layers:

- **JSON level (batch).** Standard JSON escapes — `"\n"` , `"\t"` , `"\""` . A real backslash in the final string is `"\\\\"` .
- **Excel level.** `\n` in a cell is a real line break — pair with `--prop wrapText=true` so Excel shows the wrap. Works in a shell-quoted prop directly (`--prop value='a\nb'`); `"\n"` inside batch JSON gives the same. When in doubt, `officecli get` the cell and compare character-for-character.

Other common pitfalls

Pitfall	Fix
`--name "foo"`	All attrs go through `--prop` : `--prop name="foo"`
Guessing a prop name	`officecli help xlsx <element>` — don't improvise
`--prop color=...` on a cell	Ambiguous — use `font.color` (text) or `fill` (bg). Also applies inside batch JSON: always use full dotted names, never shell aliases
`#FF0000` hex colors	Drop the `#` : `FF0000`
`--index` vs `[N]`	`--index` is 0-based (array); `[N]` paths are 1-based (XPath)
Unquoted `[N]` in zsh/bash	Quote every path: `"/Sheet1/row[1]"`
Sheet name with spaces	Quote full path: `"/My Sheet/A1"`
Year showing as `2,026`	`--prop type=string` or `numFmt="@"`
Modifying a file open in Excel	Close it in Excel first
`swap` not reordering sheets	`swap` is for rows/cells. Use `move --after` / `--before` / `--index` for sheets
Cached values missing after write	New formulas get cached values when a human opens the file; `validate` accepts them

	either way